

PD-1: RSVPs with a waitlist

Status: tickets approved · Revision 3 · Created by Ivan Blažević <ivan.blazevic@rubycode.co> · 29 September 2026

Overview

What

Signed-in people can RSVP to an event and cancel their RSVP. The event page shows how many seats are left. When an event is full, RSVPing puts you on a waitlist; when someone cancels, the first person waiting gets the seat. The organizer sees who is going and who is waiting.

Why

Events have a capacity, but nothing counts seats. Organizers collect names in chat and the small rooms overflow.

Where

The event page (`/events/:id`): an RSVP card in the sidebar, and an attendee list for the organizer.

Who

Ivan Blažević, Gather team

When

Before Ruby Zagreb Meetup #42

Background

Rails 8 authentication is in place; `Current.user` is the signed-in person.

Existing Data Structure

Nothing in the schema stores RSVPs, attendance or seat usage today. `events.capacity` is only displayed. The sidebar in `app/views/events/show.html.erb` renders it as `pluralize(@event.capacity, "seat")`, and `app/views/events/index.html.erb` shows it on the listing.

Event — `app/models/event.rb` (table `events`)

- Columns: `id` integer not null, `title` string not null, `description` text, `starts_at` datetime not null, `venue` string not null, `capacity` integer not null, `organizer_id` integer not null, `created_at` / `updated_at` datetime not null.
- Indexes: `index_events_on_organizer_id`, `index_events_on_starts_at`. Foreign key `organizer_id` → `users.id`.
- Associations: `belongs_to :organizer, class_name: "User"`.
- Validations: presence of `title`, `starts_at`, `venue`; `capacity` must be an integer greater than 0.
- Behaviour: `scope :upcoming ( starts_at >= now, ordered by starts_at )` and `organized_by?(user)`, which is the only authorization check in the app today.
- Controller: `app/controllers/events_controller.rb`. `index` and `show` allow unauthenticated access and call `resume_session`. `edit` and `update` go through `require_organizer`. There is no `destroy` route (`resources :events, except: :destroy`).

User — `app/models/user.rb` (table `users`)

- Columns: `id` integer not null, `email_address` string not null (unique index), `password_digest` string not null, `name` string not null (default `' '`), `created_at` / `updated_at` datetime not null.
- Associations: `has_many :sessions, dependent: :destroy`; `has_many :organized_events, class_name: "Event", foreign_key: :organizer_id, inverse_of: :organizer, dependent: :destroy`.

Session — `app/models/session.rb` (table `sessions`)

- Columns: `id` integer not null, `user_id` integer not null (indexed, foreign key to `users`), `ip_address` string, `user_agent` string, `created_at` / `updated_at` datetime not null.
- Associations: `belongs_to :user`.
- Relevance: `Current.user` is resolved through `Current.session`, which is set by `app/controllers/concerns/authentication.rb`. This change reads it and does not modify it.

Architectural changes

Target design

A new `Rsvp` record joins one `User` to one `Event`. Its `status` is either `going` or `waitlisted`. Seat state is always derived from these rows and never stored on `events`:

- `seats_taken = event.rsvps.going.count`
- `seats_left = [event.capacity - seats_taken, 0].max`
- Waitlist order is `created_at`, `id` among `waitlisted` rows. A person's position is 1 plus the number of waitlisted rows created before theirs.

Data flow

1. **RSVP.** `POST /events/:event_id/rsvp` goes to `RsvpsController#create`, which calls `@event.rsvp(Current.user)`. Inside one transaction the method:
2. returns the existing RSVP if the user already has one, so repeated clicks do nothing;
3. otherwise creates the RSVP as `going` when `seats_taken < capacity`, and as `waitlisted` when the event is full.

The controller then redirects back to the event with a notice saying "You're going" or "You're on the waitlist (#N)".

1. **Cancel.** `DELETE /events/:event_id/rsvp` goes to `RsvpsController#destroy`, which finds `Current.user.rsvps.find_by(event: @event)` and calls `destroy`. The `Rsvp` model has an `after_destroy` callback that runs only when the destroyed row was `going`. It promotes the oldest `waitlisted` RSVP for that event to `going`. The callback runs inside the same transaction as the `destroy`, so the freed seat and the promotion commit together.
2. **Show page.** `EventsController#show` keeps its current shape: a server-rendered page using Turbo Drive with no custom JavaScript. It also loads the current user's RSVP and, for the organizer only, the ordered RSVP list with users preloaded. The sidebar renders two new partials: an RSVP card and an attendee list.

Concurrency

Production uses SQLite (`config/database.yml`). The app runs Rails 8.1, where SQLite transactions default to `IMMEDIATE`, so only one writer holds the database at a time. That makes count-then-insert inside a transaction safe against two people taking the last seat together; this must be confirmed by a test (see Testing). The code should also call `event.lock!` at the start of the transaction. On SQLite it does nothing, but it keeps the logic correct if the app ever moves to PostgreSQL. A unique index on `(event_id, user_id)` stops duplicate RSVPs regardless of locking.

Placement

The logic lives on the models (`Event#rsvp`, `Event#seats_left`, the `Rsvp` callback), in keeping with the current small, model-centric codebase. The app has no services directory and no policy gem, so none are introduced.

Legacy coexistence

Existing events have no `rsvps` rows, so they show `seats_left == capacity`, which is correct because nobody has RSVPed through the app. Names collected in chat are not imported (see Outstanding questions). There is no older column to dual-write or backfill. The rollout is expand-only, and nothing is contracted.

Database changes

Step 1 — Expand: create `rsvps` (the only schema change)

Migration file: `db/migrate/<timestamp>_create_rsvps.rb`.

Column	Type	Null	Default	Notes
<code>id</code>	integer	not null	auto	primary key
<code>event_id</code>	integer	not null	—	<code>t.references :event, null: false, foreign_key: true</code>
<code>user_id</code>	integer	not null	—	<code>t.references :user, null: false, foreign_key: true</code>
<code>status</code>	string	not null	none	'going' or 'waitlisted'
<code>created_at</code>	datetime	not null	—	also used to order the waitlist
<code>updated_at</code>	datetime	not null	—	

Indexes and constraints:

- Unique index on `[:event_id, :user_id]` (`index_rsvps_on_event_id_and_user_id`), so a person has at most one RSVP per event.
- Index on `[:event_id, :status, :created_at]` (`index_rsvps_on_event_id_and_status_and_created_at`). It serves the seats-taken count, the waitlist head lookup and waitlist position counts.
- Index on `user_id`, created by `t.references`.
- Check constraint `status IN ('going', 'waitlisted')` via `t.check_constraint`, named `rsvps_status_check`.
- Foreign keys to `events` and `users` without `ON DELETE CASCADE`. Deletes go through model `dependent: options` so the waitlist-promotion callback runs where it matters.

`status` has no default on purpose. The model must always decide between `going` and `waitlisted` explicitly.

Later steps

- **Dual write:** not applicable, because no existing column holds this data.
- **Backfill:** none. Existing events start with zero RSVPs.
- **Switch reads:** not applicable. The seats-left display reads `rsvps` from day one.
- **Contract:** none. No columns are removed or renamed, so no `ignored_columns` step is needed.

Rollback

`drop_table :rsvps` is reversible. If it is run after launch, all RSVPs are lost, so rolling back after people have RSVPed needs a DB snapshot first.

Application changes

Rsvp **model** — `app/models/rsvp.rb` **(new)**

- `belongs_to :event, belongs_to :user`.
- `enum :status, %w[ going waitlisted ].index_by(&:itself), validate: true`.
- `validates :user_id, uniqueness: { scope: :event_id }`. The unique index is still the real guarantee.
- `scope :in_line_order, -> { order(:created_at, :id) }`.
- `after_destroy :promote_next_waitlisted, if: :going?`. It runs `event.rsvps.waitlisted.in_line_order.first&.going!`.
- `waitlist_position`: returns `nil` unless `waitlisted?`. Otherwise it returns 1 plus the count of earlier waitlisted rows for the same event.

Event **model** — `app/models/event.rb`

- `has_many :rsmps, dependent: :delete_all`. When an event is deleted there is nobody to promote, so callbacks are skipped on purpose.
- `has_many :attendees, -> { merge(Rsvp.going) }, through: :rsmps, source: :user` (optional; handy for views and tests).
- `seats_taken, seats_left, full?`.
- `rsvp_for(user)`: returns `nil` for a guest.
- `rsvp(user)`: `` ruby transaction do lock! rsmps.find\_by(user: user) || rsmps.create! (user: user, status: full? ? :waitlisted : :going) end ``
- `Rescue ActiveRecord::RecordNotUnique` and return the existing row.
- Reject events whose `starts_at` is in the past. This depends on the answer under Outstanding questions.
- No change to the `capacity` validation. Changing capacity after RSVPs is out of scope; see Risks.

User **model** — `app/models/user.rb`

- `has_many :rsmps, dependent: :destroy`. Deleting a user frees their seat and promotes the next person in line.

Routes — `config/routes.rb`

```
resources :events, except: :destroy do
  resource :rsvp, only: %i[ create destroy ]
end
```

This adds `event_rsvp_path(event)` for `POST` and `DELETE /events/:event_id/rsvp`.

RsvpsController — app/controllers/rsvps_controller.rb (new)

- Inherits `require_authentication` from `ApplicationController`, so guests are redirected to sign-in.
- `before_action :set_event ( Event.find(params[:event_id]) )`.
- `create`: `rsvp = @event.rsvp(Current.user)`, then `redirect_to @event` with notice "You're going." or "You're #N on the waitlist."
- `destroy`: `Current.user.rsvps.find_by(event: @event)&.destroy`, then `redirect_to @event` with notice "Your RSVP was cancelled." It is idempotent when no RSVP exists.
- No strong params. The user always comes from `Current.user` and the event from the URL.

Authorization

The app has no policy layer; `Event#organized_by?` is the existing check. Rules:

- Any signed-in user can create or destroy only their own RSVP.
- The attendee list renders only when `@event.organized_by?(Current.user)`.

Whether the organizer can RSVP to their own event is an open question.

EventsController#show — app/controllers/events_controller.rb

- Add `@rsvp = @event.rsvp_for(Current.user)`.
- When the organizer is viewing: `@rsvps = @event.rsvps.includes(:user).in_line_order`.
- `index`, `edit`, `update` and `create` do not change.

Views

- `app/views/events/show.html.erb`: replace the "Capacity" row with "Seats left: N of M" ("Full · N waiting" when full). Render `events/_rsvp_card` in `aside#event-facts`. Render `events/_attendees` below it for the organizer.
- `app/views/events/_rsvp_card.html.erb` (new):
- Guest: "Sign in to RSVP" link to `new_session_path`.
- No RSVP: `button_to "RSVP"`, or "Join waitlist" when full, posting to `event_rsvp_path(@event)`.
- Going: "You're going" plus `button_to "Cancel RSVP"`, `event_rsvp_path(@event)`, `method: :delete`.
- Waitlisted: "You're #N on the waitlist" plus "Leave waitlist".
- `app/views/events/_attendees.html.erb` (new): "Going (N)" as an ordered list of `user.name`, and "Waitlist (N)" with positions. Names only, no email addresses.
- `app/assets/stylesheets/application.css`: small styles for the card and lists, reusing the existing `card`, `button` and `muted` classes.

Known gap

If a guest is sent to sign-in and then signs in, they land on the root page, not back on the event. `request_authentication` only stores `return_to` for GET redirects it triggers itself. Fixing this needs a small change to how `return_to` is stored; see Outstanding questions.

Infrastructure changes

None of the following are needed:

- **Queues or jobs:** none. Promotion happens synchronously inside the cancel transaction, and email notifications are out of scope.
- **External services:** none.
- **Feature flags:** none. The repository has no feature-flag library. The feature ships to everyone once the UI is deployed.
- **Database:** one new SQLite table, created by the normal deploy migration step.

Rollout order (two deploys):

1. Deploy the migration plus the `Rsvp`, `Event` and `User` model changes. Nothing is visible to users yet.
2. Deploy the routes, `RsvpsController`, `EventsController#show` changes and views.

Both deploys must land and be checked before Ruby Zagreb Meetup #42 opens for RSVPs. That event's date isn't given (see Outstanding questions).

Work overview

Out of Scope

Email notifications, paid tickets, guests (+1), and changing capacity after people have RSVPed.

Work items

#	Title	Type	Kind	Estimate	Depends	Issue
T1	Migration: Create rsvps table	TASK	migration	1	-	#3
T2	Add Rsvp model and Event rules for RSVP, cancel, waitlist and promotion	TASK	code	5	T1	#4
T3	Let signed-in people RSVP, join the waitlist and cancel from the event page	STORY	code	5	T2	#5
T4	Show the organizer who is going and who is waiting	STORY	code	3	T3	#6
T5	Show seats left on the events index	STORY	code	2	T2	#7

Estimated total: 16 points

T1. Migration: Create rsvps table

Add the new rsvps table, which links one user to one event with a status of 'going' or 'waitlisted'. This is the only schema change for PD-1. It only adds a table, so it can ship before any code reads or writes it. No model, route or UI is part of this ticket. Columns: id; event_id integer not null with a

foreign key to events; user_id integer not null with a foreign key to users; status string not null with no default; created_at and updated_at datetime not null. Add a unique index on [event_id, user_id], a composite index on [event_id, status, created_at], the user_id index that t.references creates, and a check constraint named rsvps_status_check with the condition status IN ('going', 'waitlisted'). The foreign keys must not use ON DELETE CASCADE, because deletes go through model dependent: options so that callbacks run.

ACCEPTANCE CRITERIA

1. Running bin/rails db:migrate creates the rsvps table with event_id, user_id, status, created_at and updated_at, all NOT NULL, and status has no default
2. Inserting a second row with the same event_id and user_id fails with ActiveRecord::RecordNotUnique through the index index_rsvps_on_event_id_and_user_id
3. Inserting a row with a status other than 'going' or 'waitlisted' fails at the database because of the check constraint rsvps_status_check
4. The indexes index_rsvps_on_event_id_and_status_and_created_at and index_rsvps_on_user_id exist
5. Inserting a row that points to a missing event or user fails the foreign key check, and neither foreign key cascades on delete
6. bin/rails db:rollback drops the rsvps table and leaves events, users and sessions unchanged

IMPLEMENTATION NOTES

db/migrate/<timestamp>_create_rsvps.rb: create_table :rsvps do |t| t.references :event, null: false, foreign_key: true, index: false t.references :user, null: false, foreign_key: true t.string :status, null: false t.timestamps t.index [:event_id, :user_id], unique: true t.index [:event_id, :status, :created_at] t.check_constraint "status IN ('going', 'waitlisted')", name: 'rsvps_status_check' end The unique index already leads with event_id, so the separate event_id index from t.references is not needed. Commit the regenerated db/schema.rb. Production is SQLite (config/database.yml), so no concurrent index options are needed. Before this runs in production, take a backup of the SQLite file, because the database path and backup plan are still open questions.

Touches: rsvps

T2. Add Rsvp model and Event rules for RSVP, cancel, waitlist and promotion

Put all of the RSVP domain logic on the models. Nothing is visible to users yet; this matches the first of the plan's two deploys. Create the new Rsvp model: belongs_to :event and :user; a string enum for status with the values going and waitlisted (validate: true); uniqueness of user_id scoped to event_id; a scope in_line_order that orders by created_at and then id; and waitlist_position, which returns nil unless the RSVP is waitlisted and otherwise returns 1 plus the number of earlier waitlisted rows for the same event. Add an after_destroy callback that runs only when the destroyed row was going. It promotes the oldest waitlisted RSVP for that event to going, inside the same transaction, and logs the event id and user id. Event gains: has_many :rsvps, dependent: :delete_all; has_many :attendees (going users); seats_taken (memoized, so a page counts only once), seats_left (never below 0), full?, waitlist_count, rsvps_open? (starts_at is still in the future), rsvp_for(user) (nil for a guest), and rsvp(user). rsvp(user) returns nil when the event has started or when the user is the organizer. Otherwise, inside a transaction that begins with lock!, it returns the user's existing RSVP

or creates one as going when seats are free and as waitlisted when the event is full. It also rescues RecordNotUnique and returns the existing row. User gains has_many :rsvps, dependent: :destroy, so deleting a user frees their seat and promotes the next person.

ACCEPTANCE CRITERIA

1. An event with capacity 3 and no RSVPs has 3 seats left, is not full, and has an empty waitlist
2. For an event with capacity 2, four people calling rsvp in turn get going, going, waitlisted #1 and waitlisted #2, and going RSVPs have a waitlist_position of nil
3. Calling rsvp twice for the same person returns the same record and changes neither the going count nor the waitlisted count
4. When several threads race for the last seat of an event, exactly one RSVP is going, the rest are waitlisted, and the going count never exceeds capacity
5. Destroying a going RSVP promotes only the oldest waitlisted RSVP to going. Destroying a waitlisted RSVP promotes nobody and moves everyone behind it up one position
6. rsvp returns nil and creates no row when the user is the event's organizer or when the event's starts_at has passed
7. Destroying a going user promotes the next waitlisted person, destroying an organizer removes their events' RSVPs without errors, and seats_left is 0 rather than negative when capacity is below the going count

IMPLEMENTATION NOTES

New file: app/models/rsvp.rb. Edited files: app/models/event.rb and app/models/user.rb (add has_many :rsvps, dependent: :destroy before the existing :organized_events association; delete_all on events then removes the RSVPs of the organizer's events). Rsvp: enum :status, %w[going waitlisted].index_by(&:itself), validate: true; after_destroy :promote_next_waitlisted, if: :going?; promote_next_waitlisted does event.rsvps.waitlisted.in_line_order.first&.tap { it.going!; Rails.logger.info(...) }. waitlist_position: event.rsvps.waitlisted.where('created_at < ? OR (created_at = ? AND id < ?)', created_at, created_at, id).count + 1. Event: def rsvps_open? = starts_at.future?; seats_taken memoized with @seats_taken ||= rsvps.going.count; seats_left = [capacity - seats_taken, 0].max. rsvp(user): return if user.nil? || organized_by?(user) || !rsvps_open?; transaction { lock!; rsvps.find_by(user:) || rsvps.create!(user:, status: full? ? :waitlisted : :going) } rescue ActiveRecord::RecordNotUnique; rsvps.find_by(user:). Inside the transaction, count fresh rather than using the memoized @seats_taken. Add an rsvp factory to spec/factories. Specs: spec/models/rsvp_spec.rb (new), spec/models/event_spec.rb and spec/models/user_spec.rb. The concurrency spec needs use_transactional_fixtures turned off for that example.

Touches: rsvps, events, users

T3. Let signed-in people RSVP, join the waitlist and cancel from the event page

I want to RSVP to an event, see how many seats are left, join the waitlist when it's full and cancel my RSVP, so that I know whether I have a seat and free it up for someone else if I can't come

Expose the T2 model rules on the event page. Routes: nest resource :rsvp, only: [:create, :destroy] under resources :events. This gives POST and DELETE on /events/:event_id/rsvp. The new RsvpsController keeps the default require_authentication, loads @event with

Event.find(params[:event_id]) (404 when the id is unknown), and never reads user_id or status from params. create calls @event.rsvp(Current.user) and redirects to the event with the notice "You're going." or "You're #N on the waitlist.". When rsvp returns nil (organizer or started event), it redirects back without creating anything. destroy does nothing once the event has started. Otherwise it destroys Current.user.rsmps.find_by(event: @event) if there is one and redirects with "Your RSVP was cancelled.". EventsController#show sets @rsvp = @event.rsvp_for(Current.user). In app/views/events/show.html.erb, replace the Capacity row with "Seats left: N of M" ("Full · N waiting" when full) and render the new events/_rsvp_card partial in aside#event-facts. The card has these states. Guest: a "Sign in to RSVP" link to new_session_path. No RSVP: an "RSVP" button, or "Join waitlist" when full. Going: "You're going" and a "Cancel RSVP" button. Waitlisted: "You're #N on the waitlist" and a "Leave waitlist" button. The organizer and anyone viewing an event that has started see no RSVP buttons. After signing in, a guest is not returned to the event; that is accepted.

ACCEPTANCE CRITERIA

1. A guest opening "Ruby Zagreb Meetup #42" with 3 seats sees "Seats left: 3 of 3" and a "Sign in to RSVP" link, and sees no "RSVP" button
2. A signed-in person who clicks "RSVP" sees "You're going", sees "Seats left: 2 of 3" and sees a "Cancel RSVP" button
3. When 3 people are going, a fourth person sees "Full" and a "Join waitlist" button, and after clicking it sees "You're #1 on the waitlist"
4. When a going person clicks "Cancel RSVP", they see "Your RSVP was cancelled.", and the person who was #1 on the waitlist sees "You're going" when they open the event
5. A waitlisted person who clicks "Leave waitlist" no longer sees a waitlist position and sees the "Join waitlist" button again
6. The organizer sees no RSVP button on their own event, and nobody sees an RSVP or cancel button on an event that has already started
7. A guest who sends POST or DELETE to /events/:id/rsvp is redirected to sign-in and nothing changes. Extra user_id or status params are ignored, and an unknown event id returns 404

IMPLEMENTATION NOTES

config/routes.rb: resources :events, except: :destroy do resource :rsvp, only: %i[create destroy] end (helper: event_rsvp_path(event)). New controller: app/controllers/rsmps_controller.rb. Edited: app/controllers/events_controller.rb#show. Views: app/views/events/show.html.erb (lines 26-27 currently render pluralize(@event.capacity, 'seat')) and new app/views/events/_rsvp_card.html.erb. The buttons use button_to event_rsvp_path(@event) (method: :delete for cancel and leave) with data: { turbo_submits_with: '...' }. Use the memoized @event.seats_taken so the facts list and the card count only once. Add small .rsvp-card styles to app/assets/stylesheets/application.css, reusing the existing card, button and muted classes. Specs: spec/requests/rsmps_spec.rb (new) and spec/requests/events_spec.rb. Cucumber: features/rsmps.feature (new), reusing the steps in features/step_definitions/common_steps.rb ("{string} organizes {string} with {int} seats", "I am signed in as", "I click"). Add features/step_definitions/rsvp_steps.rb with steps such as "{string} has RSVPed to {string}" (calls event.rsvp(person(name))) and "{string} starts in the past". Use page.driver.submit for the guest POST/DELETE scenario.

Touches: rsmps, events

T4. Show the organizer who is going and who is waiting

I want to see, as the organizer, the ordered lists of people who are going and people on the waitlist, so that I know who to expect without collecting names in chat

When the signed-in user is the organizer (`@event.organized_by?(Current.user)`), `EventsController#show` also loads `@rsvps = @event.rsvps.includes(:user).in_line_order`. The new partial `app/views/events/_attendees.html.erb` renders below the RSVP card in the sidebar. It shows a "Going (N)" heading followed by an ordered list of user names, and a "Waitlist (N)" heading followed by names with their positions (`#1`, `#2`, ...). The list is read-only: there are no remove or reorder actions. It shows names only, never email addresses. Nobody else, including guests and other signed-in users, gets the list, and it is not loaded for them.

ACCEPTANCE CRITERIA

1. Organizer Ana sees "Going (3)" followed by the three attendees' names in the order they RSVPed
2. Organizer Ana sees "Waitlist (1)" with "#1" next to the name of the person who is waiting
3. After a going attendee cancels, Ana sees the promoted person under Going and the waitlist count goes down by one
4. Marko, who is signed in but is not the organizer, sees neither the "Going" nor the "Waitlist" list, and sees no other attendee's name
5. A guest sees neither the "Going" nor the "Waitlist" list
6. No attendee's email address appears anywhere on the event page, including for the organizer
7. The organizer's list shows no controls to remove or reorder people

IMPLEMENTATION NOTES

Edited: `app/controllers/events_controller.rb#show`; only query `@rsvps` inside `if @event.organized_by?(Current.user)`. New partial: `app/views/events/_attendees.html.erb`; partition `@rsvps` in memory with `@rsvps.select(&:going?)` and `.select(&:waitlisted?)`, so rendering adds no queries. Use the index in the list for waitlist positions instead of calling `waitlist_position` per row, which would be `N+1`. Render the partial from `show.html.erb` inside `aside#event-facts`, below the card. Specs: `spec/requests/events_spec.rb` (organizer sees names, non-organizer does not, no email addresses; assert with a regex on the fixture addresses). Cucumber: add scenarios to `features/rsvps.feature` using the steps from T3.

Touches: rsvps, users

T5. Show seats left on the events index

I want to see on the events listing how many seats each upcoming event has left, so that I can tell which events are filling up before I open them

The events listing (`app/views/events/index.html.erb`) currently shows `pluralize(event.capacity, 'seat')` in each event card. Replace this with the number of seats left, for example "2 of 3 seats left", or "Full" when no seats are left. `EventsController#index` loads every going count in one grouped query, `@going_counts = Rsvp.going.where(event: @events).group(:event_id).count`, so each event card adds no query. Events without RSVPs count as 0 taken. Seats left never go below 0.

ACCEPTANCE CRITERIA

1. An upcoming event with capacity 3 and no RSVPs shows "3 of 3 seats left" on the events page
2. After one person RSVPs, the events page shows "2 of 3 seats left" for that event
3. An event where every seat is taken shows "Full" on the events page
4. Waitlisted RSVPs do not reduce the seats left shown on the events page
5. An event whose capacity was lowered below its going count shows "Full" and never a negative number
6. Rendering the events page with several events runs a single query against rsvps, however many events are listed

IMPLEMENTATION NOTES

Edited: `app/controllers/events_controller.rb#index` and `app/views/events/index.html.erb` (line 16). Add a helper such as `seats_left_label(event, taken)` in `app/helpers/events_helper.rb`, next to `event_date_badge` and `event_time`, that computes `[event.capacity - taken, 0].max`. Share the wording with the show page if it fits. In the view: `seats_left_label(event, @going_counts.fetch(event.id, 0))`. Specs: `spec/requests/events_spec.rb`, counting rsvps queries with `ActiveSupport::Notifications 'sql.active_record'`. Cucumber: a `features/rsvps.feature` or `features/events.feature` scenario using "When I visit the events page".

Touches: rsvps

Risks

- **Overbooking from simultaneous RSVPs for the last seat.**
- Count and insert run in one transaction. Rails 8.1 SQLite transactions are `IMMEDIATE`, so writers are serialized, and `event.lock!` keeps this correct on other databases.
- A concurrency spec asserts that `going` never exceeds `capacity`.
- **Duplicate RSVPs from double clicks or two tabs.**
- Unique index on `(event_id, user_id)`, plus `find_by || create!` and a rescue of `RecordNotUnique`.
- `button_to` gets `data: { turbo_submits_with: ... }` so the button is disabled while submitting.
- **The waitlist doesn't advance after a cancellation.**
- Promotion is an `after_destroy` callback on `Rsvp` in the same transaction, so every destroy path (controller, user deletion) triggers it.
- Event deletion uses `delete_all` on purpose, because there is nobody left to promote.
- **The organizer lowers capacity below the number already going** (capacity changes are out of scope).
- Existing `going` rows are left as they are. `seats_left` is clamped at 0, and no one is promoted until the going count drops below capacity.
- If capacity is raised, waitlisted people are not auto-promoted in this plan.
- Whether to block capacity edits once RSVPs exist is an outstanding question.

- **Legacy events whose organizers already collected names in chat** show every seat as free, so the room can still overflow.
- Organizers need to ask attendees to RSVP in the app. Whether to import names is an outstanding question.
- **Waitlisted people are promoted without being told** (email is out of scope).
- The show page tells them their status the next time they look. This is an accepted limitation of this phase.
- **SQLite busy errors under load** (`ActiveRecord::StatementTimeout` or `SQLite3::BusyException`).
- `timeout: 5000` is already set, and the transactions are short.
- Watch the error logs (see Monitoring).

Performance

- **Event page for a guest or non-organizer:** the current event and organizer queries, plus one `COUNT` for going RSVPs and one `find_by` for the current user's RSVP. A waitlisted user adds one more `COUNT` for their position. That is about 2–3 extra queries, and each one is covered by an index.
- **Event page for the organizer:** one more query, `rsmps.includes(:user).in_line_order` . That is two queries in total (rsmps and users), with no N+1 while rendering names.
- **Keep the counts to one per render.** Memoize `seats_taken` inside `Event` , or compute it once in the view. The card and the facts list would otherwise each call `seats_taken` separately.
- **Indexes:**
- `[event_id, status, created_at]` covers the going count, the waitlist head (`ORDER BY created_at, id LIMIT 1`) and position counts.
- `[event_id, user_id]` (unique) covers the current user's lookup.
- `user_id` covers `User#rsmps` on user deletion.
- **Table size:** small. Rows are bounded by the sum of event capacities plus waitlists, likely thousands of rows. No backfill, so no batch size is needed.
- **Write path:** `create` is one lock, one count and one insert. `destroy` is one delete, and when a going RSVP is cancelled, one select plus one update for the promotion.
- **Events index:** unchanged, and it still shows `capacity` . Showing seats left there later would need a single grouped count (`Rsvp.going.where(event: events).group(:event_id).count`) or a counter cache, not a count per row.

Security

- **Authentication:** `RsvpsController` keeps the default `require_authentication` , so guests cannot create or destroy RSVPs. `EventsController#show` stays public.
- **Authorization and tampering:**
- The RSVP user is always `Current.user` . The controller accepts no `user_id` or `status` params, so nobody can RSVP or cancel for someone else, or set themselves to `going` .
- `destroy` is scoped through `Current.user.rsvps` .
- **Data exposure:**

- The attendee list and waitlist are rendered only when `@event.organized_by?(Current.user)`.
- The list shows `users.name` only. Email addresses are never rendered.
- Other visitors see aggregate counts and their own status only.
- **Input validation:** `event_id` comes from the route via `Event.find`, and an unknown id returns 404. `status` is restricted by the enum and a DB check constraint.
- **CSRF:** the forms use `button_to` with Rails' default authenticity token. No GET route changes state.
- **Brakeman:** already in the Gemfile, and it should pass on the new controller.

Risk Level: LOW — the change is additive, people act only on their own records, and the only personal data exposed (attendee names) is shown only to the event's organizer.

Monitoring

- **Errors:** watch production logs or the error tracker for exceptions in `RsvpsController#create` and `#destroy`, especially:
 - `ActiveRecord::RecordNotUnique` (should be rescued, so any occurrence is a bug);
 - `SQLite3::BusyException` and `ActiveRecord::StatementTimeout` (lock contention);
 - `ActiveRecord::RecordInvalid`.
- **Invariant check:** run it in the Rails console after launch and on the day of Meetup #42. It should always return an empty list:

```
ruby
Event.joins(:rsvps).merge(Rsvp.going).group(:id).having('COUNT(rsvps.id) >
events.capacity').pluck(:id)
```
- **Stuck waitlist check:** find events where `seats_left > 0` and waitlisted rows still exist. This should be empty unless capacity was edited.
- **Usage:**
 - RSVP create and destroy counts in the request logs (`POST` and `DELETE /events/:id/rsvp`), plus the response status mix.
 - Show-page response time on `/events/:id` before and after release.
- **Jobs:** none, because promotion is synchronous.
- **Logging:** add `Rails.logger.info` lines when a user is promoted from the waitlist (event id, user id). This makes it possible to answer questions like "why did I get a seat?".

Outstanding questions

Decided

- **Organizer RSVPs:** the organizer cannot RSVP to their own event. `Event#rsvp` rejects the call when `organized_by?(user)` is true. The RSVP card shows the organizer no RSVP button, and `RsvpsController#create` redirects back to the event without creating a row.
- **When RSVP closes:** RSVP and cancel both close when the event starts (`starts_at`). Until then, cancelling a `going` RSVP promotes the first person on the waitlist, with no earlier cutoff. From `starts_at` on, `create` and `destroy` do not change anything, and the card shows neither button.

- **What cancelling does:** cancelling deletes the `rsvps` row. There is no `cancelled` status, and `status` stays limited to `going` and `waitlisted`.
- **Capacity edits:** these stay as planned. `EventsController#update` does not block `capacity` edits once RSVPs exist, and `Event` gets no new validation. If capacity drops below the going count, the extra `going` rows stay and `seats_left` is clamped at 0. If capacity goes up, nobody on the waitlist is promoted automatically.
- **Events index:** seats left also show on `app/views/events/index.html.erb`. `EventsController#index` loads the going counts with one grouped query (`Rsvp.going.where(event: @events).group(:event_id).count`), not one count per event.
- **Managing attendees:** the organizer cannot remove or reorder attendees or people on the waitlist. The attendee list is read-only.
- **After sign-in:** a guest who signs in from the RSVP card is not returned to the event. They land where sign-in sends them today, and `return_to` handling in `app/controllers/concerns/authentication.rb` stays as it is.

Still open (not blocking)

- Should names already collected in chat for Ruby Zagreb Meetup #42 be entered as RSVPs? Those people don't have accounts, so who would enter them, and how?
- What is the date of Ruby Zagreb Meetup #42? That date sets the ship deadline.
- Where does the production SQLite database live? The path is commented out in `config/database.yml`. Will a backup be taken before the migration runs?
- Which error tracker or log destination does production use for the checks under Monitoring?

Testing

The tests use the existing RSpec, FactoryBot and Cucumber setup: `spec/`, `features/`, and the `sign_in` helper in `spec/support/authentication_helpers.rb`.

Model — `spec/models/rsvp_spec.rb` (new)

- Requires an event, a user and a valid status. An invalid status is rejected by the enum and the DB check constraint.
- A second RSVP for the same user and event fails validation, and fails at the DB unique index when validation is skipped.
- `waitlist_position` is 1, 2, 3 in `created_at` order, and `nil` for `going`.
- Destroying a `going` RSVP promotes the oldest waitlisted RSVP. Only one person is promoted.
- Destroying a `waitlisted` RSVP promotes nobody, and the positions behind it shift up.
- Destroying a `going` RSVP with an empty waitlist only frees a seat.

Model — `spec/models/event_spec.rb`

- A legacy event with no RSVPs has `seats_left == capacity` and `full? == false`.
- `rsvp(user)` gives `going` until capacity is reached, then `waitlisted`.
- `rsvp(user)` is idempotent: calling it twice returns the same record and does not change counts.
- `seats_left` never goes below 0 when `capacity` is lower than the going count.

- Concurrency: several threads RSVP for the last seat. Exactly one ends up `going` and the rest are `waitlisted`.
- The organizer and past-event rules follow whatever is decided under Outstanding questions.

Model — `spec/models/user_spec.rb`

- Destroying a user who is `going` promotes the next waitlisted person.
- Destroying an organizer removes their events' RSVPs without errors.

Request — `spec/requests/rsvps_spec.rb` (new)

- Guest `POST` and `DELETE` redirect to `new_session_path` and create nothing.
- Signed-in `POST` creates a `going` RSVP and redirects to the event with a notice.
- On a full event it creates a `waitlisted` RSVP with the position in the notice.
- `DELETE` removes only the current user's RSVP. Extra `user_id` or `status` params are ignored. `DELETE` with no RSVP redirects without error.
- An unknown `event_id` returns 404.

Request — `spec/requests/events_spec.rb`

- Show as a guest shows seats left and "Sign in to RSVP".
- Show as a signed-in user shows their status.
- Show as the organizer includes attendee and waitlist names.
- Show as a non-organizer does not include other people's names.
- No email addresses are rendered anywhere.

System / Cucumber — `features/rsvps.feature` (new), building on Given "Ana Kovač" organizes "Ruby Zagreb Meetup #42" with 3 seats :

- Three people RSVP, so the page shows "Full". A fourth person joins the waitlist as #1.
- One attendee cancels, and the waitlisted person now sees "You're going".
- Ana sees the Going and Waitlist lists in order.
- Marko, who is not the organizer, does not see the lists.
- A guest sees seats left but no RSVP button.

Sign-off

Role	Decision	By	When
Review	approved	Ivan Blažević <ivan.blazevic@rubycode.co>	29 Sep 2026 21:32